KAGGLE COMPETITION REPORT

MCTA 4363:Deep Learning

LLMs: You Can't Please Them All

LECTURERS: Dr. Hasan Firdaus Bin Mohd Zaki

GROUP MEMBER:

NAME	MATRIC NO.
AHMAD HAFIZI BIN ISHAK	2111775
LUQMAN AZFAR BIN AZMI	2219857
AFIQ ASRI	2212541

.

Table of Contents

Executive Summary

This Assessment 3 report presents an improved version of the earlier Kaggle LLM judge disagreement project. The previous assessment demonstrated the basic idea of generating adversarial essays and simulating disagreement among multiple judges. Assessment 3 extends the work into a more reproducible deep learning/NLP project by using a real judge ensemble framework, configurable providers, experiment logs, visual outputs, and a Kaggle-style submission file.

The implemented system generates several candidate essays for each topic, evaluates each candidate using multiple judge personas, calculates disagreement metrics, and selects the best essay according to an objective function. The project supports three judge modes: a heuristic baseline, Google Gemini as a real LLM judge, and local Ollama-based models such as Mistral and Qwen. The strongest disagreement in the provided experiments came from the Gemini judge run, which achieved an objective score of 7.692 and a disagreement range of 6.000 for the selected candidate.

1. Introduction

Large Language Models (LLMs) are increasingly used to generate and evaluate written content. However, judging essays is not purely objective. Different evaluators may prefer different writing styles, tones, structures, and levels of creativity. The Kaggle competition LLMs - You Cannot Please Them All focuses on this problem by rewarding systems that can create disagreement among LLM judges while still producing relevant essays.

This project studies the robustness of LLM-as-a-judge systems through an adversarial essay generation pipeline. Rather than generating only one safe academic essay, the system generates multiple candidates using different writing strategies. These candidates are then evaluated by several judge personas, and the candidate with the highest disagreement-oriented objective score is selected for submission.

2. Problem Understanding

The main challenge is that the hidden Kaggle judging system is not available locally. Therefore, a local approximation is required. The project treats each essay as a candidate response to a topic and uses several judge personas to simulate or approximate evaluator disagreement.

The key research question is: How can essay candidates be generated and selected so that different LLM judges evaluate the same essay differently while the essay remains relevant and acceptable?

- Input: Kaggle-style topic data containing an id and topic.
- Output: A submission.csv file containing one selected essay per topic.
- Evaluation focus: judge disagreement, mean score, score range, standard deviation, word count, and objective score.
- Constraint: the hidden Kaggle judges cannot be accessed locally, so Gemini/Ollama are used as practical approximations.

3. Baseline

The baseline method uses a no-API heuristic judge. It scores essays using measurable text features such as relevance to the topic, structure, length, lexical diversity, and balance markers. This baseline is useful for debugging because it is fast, free, and reproducible.

However, the heuristic judge is limited because it is handcrafted and cannot fully represent the subjective preferences of real LLM judges. Therefore, Assessment 3 compares the heuristic baseline with real judge providers such as Gemini and local Ollama models.

4. Proposed Method

The proposed method generates multiple candidate essays per topic. Each candidate uses a different writing strategy so that judge personas may disagree about which style is strongest. The candidate styles include balanced academic writing, creative reflection, concise direct argument, skeptical logic, pro/con contrast, structured headings, empathetic social framing, and uncertainty-aware reasoning.

- Candidate generation creates diverse essay styles for the same topic.
- Judge personas evaluate each candidate independently.
- A summary table calculates mean score, standard deviation, minimum score, maximum score, score range, and objective score.
- The best candidate is selected using a disagreement-oriented objective function with quality and length safeguards.

4.1 Judge Personas

Table 1: Judge persona design

Judge Persona	Preference
strict_academic	Logical structure, clarity, relevance, balanced argument, precise wording
creativity_friendly	Originality, engaging style, memorable phrasing, personal voice
concision_friendly	Concise, direct, readable answers with minimal repetition
skeptical_logic	Careful reasoning, stated limitations, avoidance of overclaiming

4.2 Selection Objective

The project selects the candidate with the highest objective score for each topic. The objective combines disagreement range, score standard deviation, a small mean score bonus, and penalties for very low quality or unsuitable length.

Selection formula

```
objective = range_score + std_score + 0.15 * mean_score - quality_penalty - length_penalty
```

This design reflects the competition goal: the selected essay should not simply be the highest-quality essay. It should be the essay that creates meaningful disagreement while avoiding extremely weak or unusable submissions.

5. System Architecture and Implementation

The Assessment 3 project is structured as a reproducible VS Code-ready pipeline. The main script loads topics, generates candidate essays, builds the selected judge provider, scores all candidates, summarizes results, selects the best candidate, exports CSV/JSONL outputs, and produces visual plots.

1. Load topic data from the data directory.
2. Generate several essay candidates for each topic.
3. Build a judge provider: heuristic, Gemini, or Ollama.
4. Run every judge persona on every candidate essay.
5. Summarize candidate scores and disagreement metrics.
6. Select the best candidate for each topic.
7. Export submission and experiment artifacts.
8. Generate visualizations for report analysis.

Table 2: Project module structure

Module	Purpose
main.py	Coordinates the full experiment pipeline and command-line arguments.
src/config.py	Loads environment variables and validates judge provider settings.
src/data.py	Loads Kaggle-style topic data and sample submission columns.
src/essay_generator.py	Creates multiple candidate essays using diverse writing strategies.
src/judges.py	Implements heuristic, Gemini, and Ollama judge providers.
src/selection.py	Calculates score summaries and selects the best candidate.
src/viz.py	Creates objective and disagreement plots.
src/export.py	Saves JSONL logs and run summaries.

5.1 Security Improvement

A key improvement from the earlier assessment is that API keys should not be hard-coded inside the report or source code. In the Assessment 3 project, configuration is designed to use a .env file and .env.example. This is safer for GitHub because private keys can be excluded from commits while the project remains reproducible for other users.

6. Experimental Setup

The experiment was executed using one sample topic with three candidate essays per topic. Each selected candidate was evaluated by four judge personas, producing twelve judgements per run. The same pipeline was tested across heuristic, Gemini, Mistral through Ollama, and Qwen through Ollama.

Table 3: Experiment run summary

Judge Provider	Run Time	Topics	Candidates	Judgements	Avg Objective	Avg Mean Score	Avg

							Disagreement Range
Heuristic	2026-06-19T22:22:35	1	3	12	3.796	8.033	1.8
Gemini	2026-06-19T22:58:35	1	3	12	7.692	3.75	6.0
Ollama - Mistral	2026-06-19T22:50:49	1	3	12	4.125	7.5	2.0
Ollama - Qwen	2026-06-19T22:46:28	1	3	12	4.045	7.25	2.0

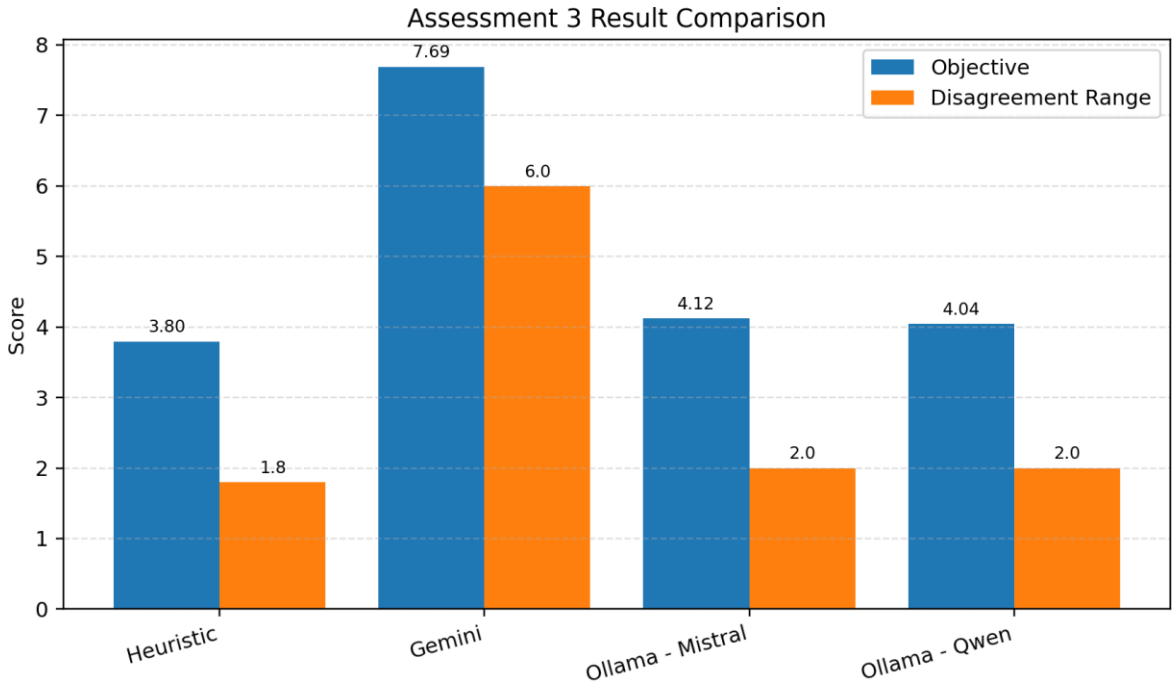
7. Results and Analysis

The table below compares the selected candidate from each judge provider. The objective score is the main optimization value. The disagreement range shows the difference between the highest and lowest judge scores for the selected essay.

Table 4: Selected candidate comparison

Judge Provider	Selected Strategy	Mean Score	Std Score	Range	Objective	Word Count
Heuristic	uncertainty_aware	8.033	0.791	1.8	3.796	152
Gemini	empathetic_social	3.75	2.63	6.0	7.692	158
Ollama - Mistral	empathetic_social	7.5	1.0	2.0	4.125	158
Ollama - Qwen	empathetic_social	7.25	0.957	2.0	4.045	158

Figure 1: Cross-provider objective and disagreement comparison



The Gemini run produced the strongest selected objective score of 7.692 and the highest disagreement range of 6.000. This indicates that Gemini judge personas reacted very differently to the selected empathetic_social essay. In comparison, the heuristic judge achieved a lower objective score of 3.796 and a disagreement range of 1.800, suggesting that handcrafted scoring produces less extreme disagreement.

The local Ollama runs using Mistral and Qwen produced moderate disagreement. Both selected the empathetic_social strategy, but their objective scores were lower than Gemini. This shows that local LLM judges can approximate real judge disagreement, although the disagreement intensity may depend strongly on the model used.

Figure 2: Gemini candidate objective scores

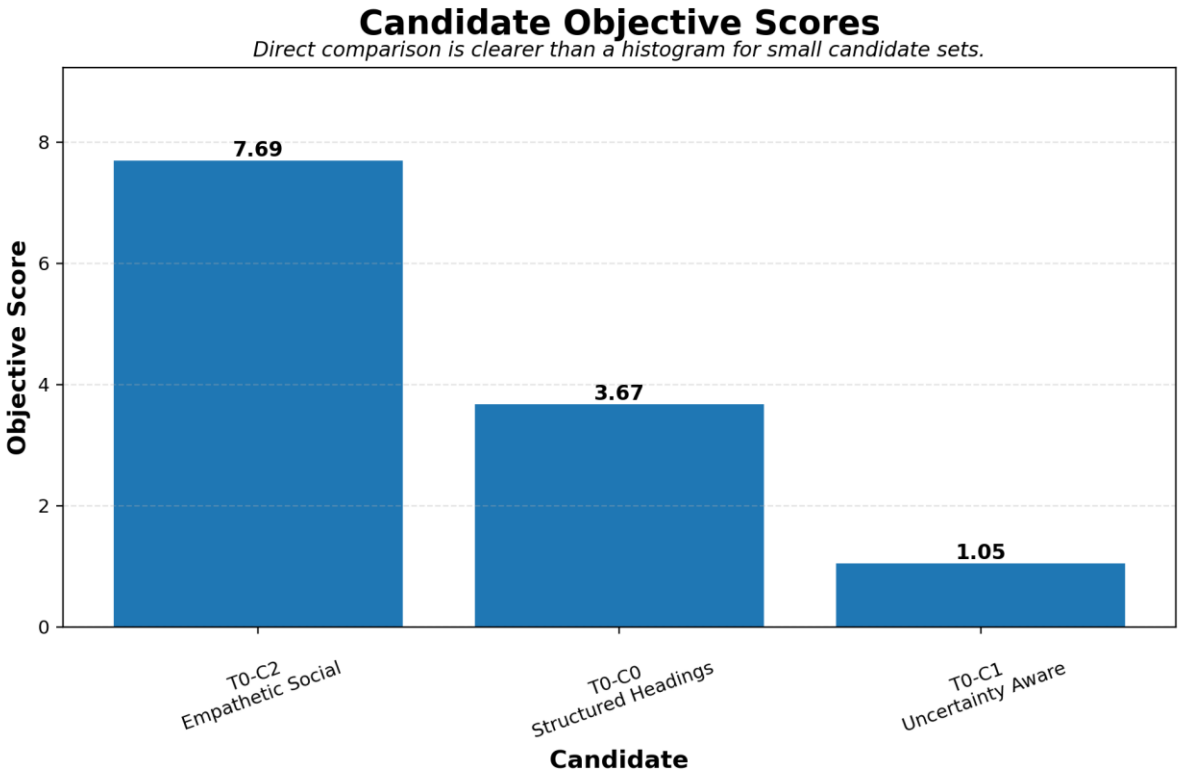
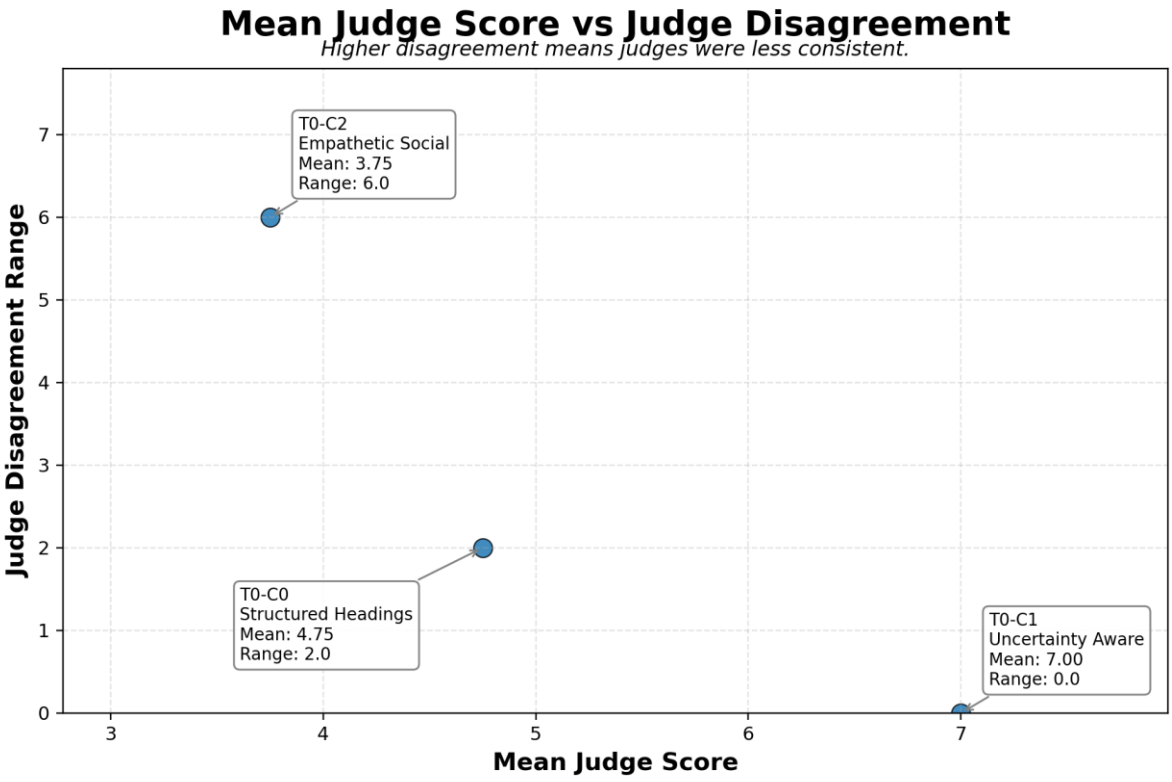


Figure 3: Gemini mean judge score versus disagreement range



7.1 Selected Essay Analysis

The Gemini run selected the empathetic_social strategy. This strategy emphasizes the human experience, fairness, exclusion, dependency, and dignity. Such language can be valued highly by creativity-friendly or empathy-oriented judges but judged more critically by strict or logic-focused judges. This explains why the disagreement range became large.

Selected Gemini essay excerpt

```
Discussions about Is technology making students more independent or more dependent? often focus on
systems and results, but the human experience is just as important. A policy or tool can be
technically impressive and still fail if people feel confused, excluded, or pressured by it. The
advantage is that the idea may help ordinary users. It can make tasks easier, support people with
fewer resources, and offer new ways to participate. When designed with care, it can make society
more open and responsive. The danger is that people may be judged by a system they do not
understand. They may also become dependent on a process that appears objective but still reflects
human assumptions. This is why transparency and education are essential. My final view is that the
idea should be accepted, but not treated as neutral or perfect. It must be shaped around human
dignity, fairness, and accountability. Progress should make people more capable, not less heard.
```

8. Discussion

The results confirm the central idea of the project: an essay can be relevant to the topic while still dividing judges. The highest disagreement was not produced by the safest academic strategy, but by the empathetic_social strategy under Gemini judging. This suggests that subjective values such as human dignity, social fairness, and emotional framing can create stronger disagreement than purely factual or formulaic writing.

A second observation is that different judge providers behave differently. The heuristic judge is stable and predictable, but less representative of real LLM evaluation. Gemini produced the highest variation, while local Ollama models produced more moderate disagreement. This supports the decision to include multiple provider modes in the project.

The project also shows why experiment logging is important. By saving all judge scores, candidate summaries, selected candidates, run summaries, and plots, the workflow becomes easier to explain in a report and easier to reproduce in GitHub.

9. Limitations

- The hidden Kaggle judge is not available locally, so Gemini and Ollama are approximations rather than exact replicas.
- Only one sample topic was processed in the provided output, so the experiment should be expanded to more topics for stronger conclusions.
- Real LLM judge scores can vary depending on model version, temperature, rate limits, prompt wording, and API availability.
- Candidate generation is mainly template-based. Future work can add LLM-generated candidates or fine-tuned generation strategies.
- API keys must be protected using .env files and should never be committed to GitHub or written in reports.

10. Conclusion

Assessment 3 successfully improves the earlier LLM judge disagreement project into a complete and reproducible NLP/deep learning workflow. The system supports candidate generation, real or simulated judge evaluation, disagreement-based selection, CSV/JSONL export, experiment summaries, and visualization. The Gemini run achieved the strongest disagreement in the provided experiment, while local Ollama models demonstrated that offline judge approximation is also possible.

Overall, the project demonstrates that LLM evaluation is not a single fixed truth. Different judges can legitimately disagree based on their priorities. A robust evaluation system should therefore record not only the average score, but also disagreement patterns, judge reasons, score range, and selection trade-offs.

References

- Kaggle. LLMs - You Cannot Please Them All competition task and data format.
- Project source files: main.py, src/config.py, src/essay_generator.py, src/judges.py, src/selection.py, src/viz.py, and src/export.py.
- Experiment artifacts: outputs_C3_heuristic, outputs_C3_gemini, outputs_C3_mistral, and outputs_C3_qwen.
- PROJECT_REPORT_TEMPLATE.md: report structure for real LLM judge ensemble project.
- Previous Assessment 2 report: baseline methodology and discussion model for LLM judge disagreement.

Appendix A: Important Output Files

Table 5: Generated project artifacts

File	Description
submission.csv	Final Kaggle-style file with topic id and selected essay.
all_judge_scores.csv	Raw score given by each judge persona for each candidate.
candidate_summary.csv	Mean score, standard deviation, range, penalties, and objective per candidate.
selected_candidates.csv	Best candidate chosen for each topic according to objective score.
all_judge_scores.jsonl	JSONL experiment log for programmatic inspection.
run_summary.md / run_summary.json	Summary of provider, candidate count, judgements, and average selected metrics.
objective_distribution.png	Chart showing candidate objective scores.
mean_vs_disagreement.png	Scatter plot showing mean judge score against disagreement range.

Appendix B: Key Implementation Snippets

Pipeline execution snippet

```
parser.add_argument("--judge", choices=["heuristic", "gemini", "ollama"], default=None)
parser.add_argument("--n-candidates", type=int, default=None)
parser.add_argument("--max-topics", type=int, default=None)


score_df = score_candidates(all_candidates, judge)
summary_df = summarize_candidates(score_df)
selected_df = select_best(summary_df)
submission = make_submission(selected_df, id_col=submission_id_col, essay_col=submission_essay_col)
```

Candidate selection snippet

```
summary["range_score"] = summary["max_score"] - summary["min_score"]
summary["quality_penalty"] = np.where(summary["mean_score"] < 4.0, 1.5, 0.0)
summary["length_penalty"] = np.where((summary["word_count"] < 90) | (summary["word_count"] > 500),
0.5, 0.0)
summary["objective"] = summary["range_score"] + summary["std_score"] + 0.15 * summary["mean_score"]
- summary["quality_penalty"] - summary["length_penalty"]
```